Reporte Técnico de Actividades Práctico-Experimentales Nro. 006

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Jaime A. Landazuri
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	06
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamán Q.
Fecha	05 y 06 de noviembre de 2026
Horario	Jueves 10h30–11h30 / Viernes 07h30–09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante pruebas de bondad de ajuste, utilizando los métodos Chi-cuadrado y Kolmogorov-Smirnov para determinar si los valores generados presentan un comportamiento compatible con una distribución uniforme en el intervalo $[0, 1]$.

1. Generar secuencias de números pseudoaleatorios utilizando el método congruencial lineal (LCG).
2. Aplicar la prueba estadística Chi-cuadrado para evaluar la uniformidad de las frecuencias observadas.
3. Aplicar la prueba Kolmogorov-Smirnov para comparar la distribución empírica con la distribución uniforme teórica.
4. Interpretar los estadísticos de prueba y sus respectivos valores p.
5. Comparar el comportamiento estadístico de diferentes configuraciones del generador pseudoaleatorio.

3. Materiales, Reactivos, Equipos y Herramientas

- Python 3.14 con NumPy y Matplotlib.
- Java (OpenJDK 27).

- Entorno de desarrollo integrado: Visual Studio Code, con compilación y ejecución desde la terminal (`python3`, `javac`/`java`).
- Scripts propios de verificación cruzada entre implementaciones (`check_parity.py`).

4. Procedimiento / Metodología Ejecutada

4.1. Generador congruencial lineal

Se implementó el generador congruencial lineal (LCG) definido por la recurrencia

$$X_{n+1} = (a \cdot X_n + c) \bmod m, \qquad U_n = \frac{X_{n+1}}{m}, \tag{1}$$

en dos lenguajes (Python y Java) con parámetros idénticos. Las configuraciones evaluadas corresponden a generadores clásicos documentados en el material de la asignatura (Semana 5):

Tabla 1. Configuraciones clásicas del LCG evaluadas: referencia documentada, módulo m, multiplicador a, incremento c y semilla inicial.

Configuración	**Referencia**	m	a	c	**Semilla** X_0
glibc	libc GNU/Linux	2^{32}	1103515245	12345	1
java_random	`java.util.Random`	2^{48}	25214903917	11	42
zx81_bad	ZX81 BASIC (mala calidad)	$2^{17}+1$	75	74	1

Con cada configuración se generaron $n = 10\,000$ números pseudoaleatorios en $[0, 1)$.

4.2. Prueba Chi-cuadrado

El intervalo $[0, 1]$ se dividió en $k = 10$ subintervalos de igual amplitud, con frecuencia esperada $E_i = n/k = 1000$ y grados de libertad $gl = k - 1 = 9$. El estadístico se computó como $\chi^2 = \sum_{i=1}^{k} \frac{(O_i - E_i)^2}{E_i}$ y el valor p mediante la función gamma incompleta regularizada implementada desde cero (mismo algoritmo en ambos lenguajes). Regla de decisión con $\alpha = 0{,}05$: rechazar H_0 si $p < 0{,}05$ (equivalentemente, si $\chi^2 > \chi^2_{1-\alpha,9} = 16{,}92$).

4.3. Prueba Kolmogorov-Smirnov

Se comparó la función de distribución acumulada empírica $F_N(x)$ con la teórica $F(x) = x$, calculando $D = \max(D^+, D^-)$ y el valor p con la distribución Kolmogorov asintótica evaluada en $\sqrt{n} \cdot D$ (coincide con la referencia de `scipy.stats.kstest`). Umbral crítico aproximado: $D_{\text{crit}} \approx 1{,}36/\sqrt{n} = 0{,}0136$.

4.4. Verificación cruzada

Se verificó que ambas implementaciones produzcan la **misma secuencia** con los mismos parámetros: los 200 primeros estados enteros y todos los estadísticos de prueba coincidieron exactamente entre Python y Java (`check_parity.py`).

5. Resultados

5.1. Frecuencias observadas por intervalo ($k = 10$, $E_i = 1000$)

Tabla 2. Frecuencias observadas por intervalo para cada configuración ($k = 10$ intervalos, frecuencia esperada $E_i = 1000$ con $n = 10\,000$ valores).

Config.	1	2	3	4	5	6	7	8	9	10
glibc	1001	973	989	1015	1005	984	1017	985	1022	1009
java_random	1036	1006	971	958	1047	961	1009	1042	961	1009
zx81_bad	968	994	1015	947	970	1039	1024	1024	1007	1012

5.2. Resumen estadístico ($n = 10\,000$, $\alpha = 0{,}05$)

Tabla 3. Resultados de la prueba Chi-cuadrado por configuración ($gl = 9$, $\alpha = 0{,}05$): estadístico, valor p y decisión estadística.

Configuración	χ^2 ($gl = 9$)	p	Decisión
glibc	2.4360	0.9826	No se rechaza H_0
java_random	11.1140	0.2680	No se rechaza H_0
zx81_bad	7.8600	0.5483	No se rechaza H_0

Tabla 4. Resultados de la prueba Kolmogorov–Smirnov por configuración ($\alpha = 0{,}05$): estadístico D, valor p y decisión estadística.

Configuración	D (KS)	p	Decisión
glibc	0.007134	0.3111	No se rechaza H_0
java_random	0.007012	0.2907	No se rechaza H_0
zx81_bad	0.011633	0.8665	No se rechaza H_0

5.3. Verificación de equivalencia Python/Java

Se generó la secuencia con cada configuración en ambos lenguajes, se volcaron los estados enteros a CSV (`results/parity_*_*.csv`) y un script comparador (`check_parity.py`) contrastó columna a columna ambos archivos, además de los estadísticos de prueba informados por cada lenguaje. Los resultados de esa comparación resumen la tabla siguiente:

Tabla 5. Evidencia de equivalencia Python/Java por configuración: comparación bit a bit de los primeros 200 estados, frecuencias, χ^2 y D ejecutada por `check_parity.py`.

Config.	Primeros 200 estados idénticos	Frecuencias idénticas	χ^2 idéntico	D idéntico	Veredicto
glibc	sí	sí	sí	sí	Idénticas
java_random	sí	sí	sí	sí	Idénticas
zx81_bad	sí	sí	sí	sí	Idénticas

Como muestra de estados enteros, los tres primeros valores de la configuración glibc fueron idénticos en ambos lenguajes: $X_1 = 1103527590$, $X_2 = 2524885223$, $X_3 = 662824084$. La equivalencia bit a bit confirma que las pruebas estadísticas del informe no dependen de particularidades de implementación de un lenguaje.

5.4. Gráficos

El análisis gráfico es coherente con las tablas: (1) en la Figura 1 la curva empírica se superpone a la diagonal teórica, sin franjas de separación apreciables; (2) en la Figura 2 las frecuencias observadas de glibc oscilan levemente alrededor de $E_i = 1000$ sin sesgo sistemático, lo que explica el valor χ^2 bajo (2,44) y su valor p alto (0,983); y (3) en la

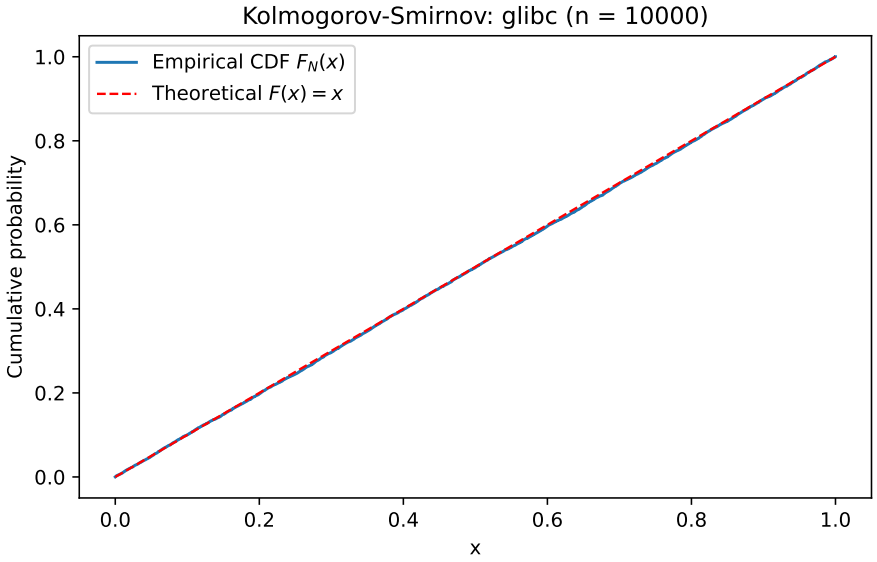


Figura 1. Prueba Kolmogorov–Smirnov para la configuración glibc: función de distribución acumulada empírica frente a la teórica $F(x) = x$ de una uniforme. La desviación máxima visual es $D = 0{,}007$, inferior al umbral crítico $\approx 0{,}0136$.

Figura 3, los pares consecutivos de glibc y java_random llenan el cuadrado $[0, 1]^2$ de manera homogénea, mientras que el generador zx81_bad exhibe la estructura reticular típica de un LCG mal parametrizado: pares concentrados en pocas líneas. Esa figura aporta exactamente la evidencia que las pruebas de uniformidad *no* capturan: el defecto de zx81_bad no es de frecuencias marginales, sino de independencia serial.

5.5. Hallazgos complementarios

El generador zx81_bad viola las condiciones de Hull–Dobell (un factor primo de m no divide a $a - 1$): su ciclo cubre únicamente $21\,845$ estados, menos del $17\,\%$ de su espacio de $131\,073$ estados. Con $n = 10\,000$ las dos pruebas de uniformidad aplicadas no lo detectan (χ^2: $p = 0{,}548$; KS: $p = 0{,}867$), aunque su muestra de $10\,000$ valores procede de un ciclo corto que repite contenido. Este hallazgo es el más instructivo de la práctica: confirma que las pruebas de bondad de ajuste miden la uniformidad *marginal* de los valores, y pueden aprobar de forma simultánea dos generadores con calidad estructural muy distinta. El diagrama de pares consecutivos (Figura 3) sí separa a los dos generadores "buenos" del "malo": la información de dependencia serial está en el orden de la secuencia, que χ^2 y KS comparados tal como se aplicaron aquí no utilizan. Refuerza además la diferencia conceptual entre valor p alto por "buena calidad" y valor p alto "de casualidad": bajo H_0 los valores p se distribuyen uniformemente, de modo que un generador perfecto podría devolver $p = 0{,}99$ o $p = 0{,}30$ con la misma probabilidad; observar p intermedios (java_random: $0{,}268$) no es señal de un peor generador, y solo valores extremadamente bajos o extremadamente cercanos a reiteración merecen revisión.

6. Preguntas de Control

1. **¿Qué es la hipótesis nula?**

   La hipótesis nula (H_0) es la afirmación que se asume provisionalmente cierta y que la prueba estadística intenta debilitar con evidencia de los datos; es el punto de partida sin efecto ni diferencia. En esta práctica, H_0: "los números pseudoaleatorios generados siguen una distribución uniforme en $[0, 1]$" ($X \sim U(0, 1)$), contra la alter-

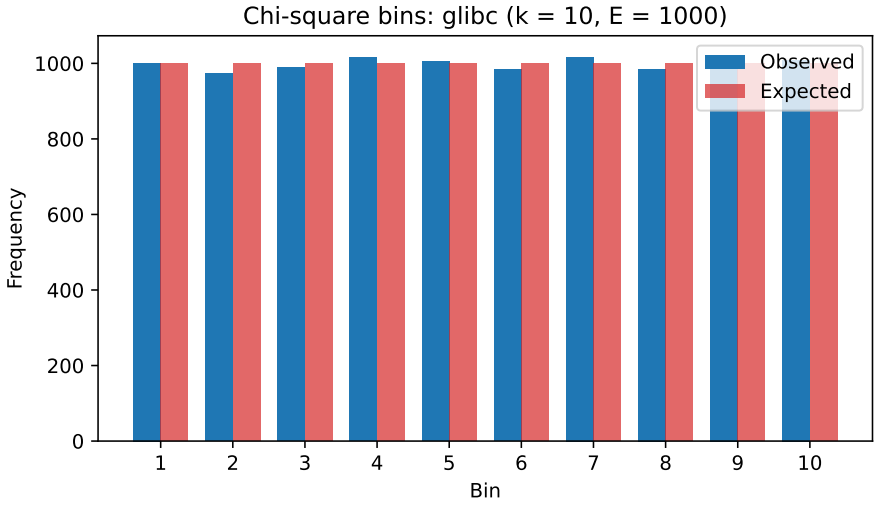


Figura 2. Vista de la prueba Chi-cuadrado para la configuración glibc: frecuencias observadas por intervalo frente a la frecuencia esperada $E_i = 1000$.

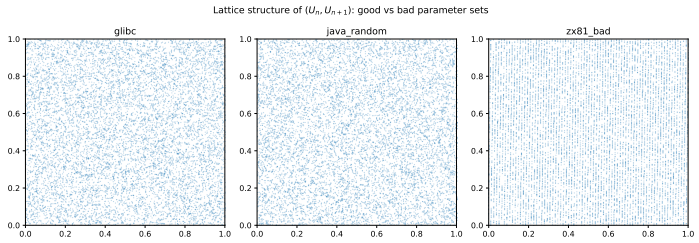


Figura 3. Diagramas de dispersión de pares consecutivos (U_n, U_{n+1}) para las tres configuraciones; se aprecia la diferencia de estructura serial entre los generadores “buenos” y el generador “malo”.

nativa H_1: “no siguen una distribución uniforme”. En los datos, H_0 es la hipótesis “de excelencia”: no afirma que el generador sea aleatorio, sino que no detectamos incompatibilidad con la uniformidad que se presupone.

2. **¿Qué representa el valor p?**

   El valor p es la probabilidad de obtener un resultado de la prueba (estadístico) al menos tan extremo como el observado, suponiendo que H_0 es cierta. No es la probabilidad de que H_0 sea verdadera ni la probabilidad de que la secuencia sea aleatoria: es una medida de compatibilidad entre los datos observados y el modelo nulo. Por ejemplo, con glibc $\chi^2 = 2{,}436$ con $gl = 9$ dio $p = 0{,}983$: si el generador fuese perfectamente uniforme, resultados tan favorables a H_0 se obtienen el $98{,}3\,\%$ de las veces, de modo que los datos son completamente compatibles con la uniformidad.

3. **¿Qué significa rechazar H_0?**

   Rechazar H_0 significa que, con el umbral de significancia elegido ($\alpha = 0{,}05$), la probabilidad de observar semejante desviación “por azar” si H_0 fuera cierta es tan baja que la evidencia apoya la alternativa: los datos son incompatibles con la uniformidad al nivel de significancia asumido. Constituye la única decisión que la prueba de Fisher permite tomar sin afirmaciones sin evidencia: rechazar sí es una conclusión, no rechazar significa únicamente “no se encontró evidencia suficiente”, que H_0 es verdadera. Con $p < 0{,}05$ se concluí que la secuencia no describe un comportamiento uniforme; con $p \geq 0{,}05$ el generador pasa la prueba.

4. **¿Qué diferencia existe entre Chi-cuadrado y KS?**

   χ^2 agrupa los valores en $k = 10$ intervalos y compara frecuencias observadas contra esperadas ($E_i = 1000$), resumiendo la desviación como $\sum(O_i - E_i)^2/E_i$. Desperdicha la información de orden de la secuencia: dos muestras con la misma estructura interna en los mismos bins reciben exactamente el mismo estadsitico. La prueba de Kolmogorov–Smirnov no agrupa: compara directamente la función de distribución acumulada empírica contra la teórica y toma la máxima diferencia absoluta ($D = \max(D^+, D^-)$), sensibilitándose a la distribución completa en vez de a los intervalos. En resumen: χ^2 es una prueba de "por intervalos" (acercamiento discreto) que depende de la división elegida; KS es una prueba estática de pérdidas por valor de "x" y del puende de corte de orden con máxima brecha acumulada. En esta práctica ambas coincidian no rechazar H_0 para las tres configuraciones, lo que da una segunda lectura de la consistencia de los resultados.

5. **¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?**

   Por tres razones: (1) las pruebas son de falsación, no de verificación: cuando mucho, detectan incompatibilidad con una propiedad concreta (uniformidad); aprobarlas solo significa "no se encontró evidencia en contra", y siempre existen infinidad de estructuras defectuosas que la prueba no mide; (2) la secuencia es determinista por construcción: un LCG es una expresión recursiva predecible; ninguna de sus propiedades se gana "por azar a posteriori'; (3) cada prueba mira una propiedad aislada con una muestra limitada: aquí χ^2 y KS miran solo uniformidad con $n = 10\,000$ y no revelaron el defecto de zx81_bad, cuyo bloque tiene ciclo corto y correlación serial fuerte pero histogramas marginales uniformes, tal como se mostró en los hallazgos complementarios. Una batería suficientemente completa (uniformidad, independencia, fugas espectrales, diámetro, salud entre pruebas) puede acrecentar la confianza, pero nunca demostrar la aleatoriedad verdadera.

7. Conclusiones

1. Las tres configuraciones evaluadas pasaron simultáneamente las dos pruebas de bondad de ajuste con $n = 10\,000$ y $\alpha = 0{,}05$: χ^2 (gl = 9) con valores de p entre 0,27 y 0,98, y KS con D entre 0,0070 y 0,0116, siempre bajo el umbral crítico 0,0136. La configuración glibc, además, produce frecuencias observadas notablemente poco dispersas, la mejor lectura dentro del intervalo tolerado.

2. La comparación de configuraciones demuestra que la calidad del LCG se juega en los parámetros: glibc ($m = 2^{32}$) y java_random ($m = 2^{48}$) complen Hull–Dobell en el estado usual de implementación, mientras que zx81_bad ($m = 2^{17} + 1$) viola un factor primo y su ciclo se limita a 21 845 estados; aun así, pasó las pruebas de uniformidad, mostrando la insuficiencia de una sola familia de pruebas estadísticas.

3. La verificación cruzada Python/Java resultó bit a bit idéntica (primeros 200 estados, frecuencias, χ^2 y D para las tres configuraciones), lo que valida que las implementaciones y los valores-decision estadísticos no dependen de las peculiaridades de un lenguaje en particular.

4. Las pruebas estadísticas miden propiedades, no aleatoriedad: χ^2 y KS buscan incompatibilidades con la uniformidad, y solo pueden rechazar; aprobarlas configura evidencia compatible, no demostrativa. La Figura 3 ejemplifica el punto de vista: el generador "malo" pasa las pruebas de frecuencias y falla visualmente en independencia serial.

5. Para simulación de eventos discretos, se concluye que un LCG con parámetros correctos es una fuente pseudoaleatoria suficiente para el alcance de la asignatura, siempre que se documente la semilla (reproducibilidad, como en el resultado de paridad bit a bit) y se combinen pruebas de uniformidad con revisiones gráficas y, en el futuro próximo, pruebas de independencia (autocorrelación, series).

8. Recomendaciones

- Complementar χ^2 y KS con pruebas de independencia sobre la misma muestra (autocorrelación serial, prueba de series ortiguos, comportamiento espectral), porque la uniformidad marginal puede coexistir con dependencia serial, como se mostró con zx81_bad.
- Repetir cada prueba con distintas semillas y tamaños de muestra: aprobar una corrida con una semilla no constituye evidencia general para el generador; varias pruebas cruzadas con semillas diferentes siguen siendo insuficientes, pero mucho menos propensas a coincidir por azar.
- Aumentar la resolución de la prueba χ^2 en futuras ejecuciones (intervalos mayores, k distintos) para detectar desviaciones que una partición grosera suaviza.
- Operativizar los experimentos: reportar siempre la semilla, el módulo, los multiplicadores y las versiones de herramienta utilizados, de modo que la secuencia completa sea reproducible, como se garantizó aquí con la paridad Python/Java.
- Para aplicaciones exigentes, aun fuera del alcance de la práctica, adoptar generadores modernos (Mersenne Twister, PCG64, xorshift), reservando el LCG clásico para docencia y para flujos que se comparan entre lenguajes.

9. Declaración de uso de Inteligencia Artificial

En este trabajo se usaron herramientas de inteligencia artificial como apoyo, no como sustituto del trabajo propio. Primero se muestra un prompt representativo de la forma en que se usó la herramienta y, a continuación, la justificación de su uso.

9.1. Prompt utilizado

El estudiante partió siempre de contenido ya elaborado (las implementaciones, las ejecuciones y los resultados propios, ya volcados a archivos) y pidió mejorarlos, no generarlos desde cero. El siguiente prompt resume la instrucción general empleada (es representativo, no una transcripción literal de todas las interacciones):

> *Implementé en Python y en Java un LCG con tres configuraciones (glibc, java_random y zx81_bad), junto con las pruebas Chi-cuadrado y Kolmogorov-Smirnov (implementadas desde cero) y ya ejecuté todo: tengo los JSON y CSV de resultados y las figuras. Necesito que me ayudes con **lo que ya tengo**, no a hacerlo desde cero:*
>
> *1. Verificación técnica: revisa mis implementaciones de la gamma incompleta regularizada y del estadístico KS comparadas con scipy.stats.kstest, y dime si hay errores de criterio o redondeo.*
>
> *2. Redacción: completa el esqueleto del informe LaTeX con el análisis de resultados y los hallazgos complementarios a partir de mis tablas y mis figuras, sin inventar números.*
>
> *3. Formato: ordena las figuras con caption y referencias cruzadas, y ajusta la bibliografía a IEEE.*
>
> *Reglas: no inventes datos, cifras ni fuentes. Todo número debe salir de mis archivos de resultados. Si algo no lo puedes verificar, dímelo.*

9.2. Justificación del uso

Herramientas utilizadas.

- **GLM-5.3-flash**, mediante un agente de código en entorno local.

Apoyo recibido del modelo.

- **Verificación técnica:** revisión de la implementación propia del p-valor de χ^2 (gamma incompleta regularizada) y del p-valor de KS (forma asintótica de Kolmogorov), contrastadas con la referencia de `scipy.stats.kstest` señalada por el estudiante.
- **Redacción:** elaboración del análisis de resultados y de los hallazgos complementarios a partir del esqueleto del informe y de los archivos de resultados generados por las ejecuciones del estudiante.
- **Formato:** corrección del código LaTeX (entornos de figura con caption, referencias cruzadas, estilo IEEE de bibliografía) y compilación del PDF.

Lo que no se hizo con IA.

- Las implementaciones del LCG, de las pruebas χ^2 y KS y del script de paridad son escritas y ejecutadas por el estudiante; los modelos trabajaron sobre esa base.
- Todos los números del informe provienen de las ejecuciones del estudiante (`results/stats_*.json`, CSV y figuras); ninguno fue inventado ni hipotéticamente sugerido por el modelo.
- Las realizaciones y decisiones estadísticas (semillas, configuraciones, umbral $\alpha = 0{,}05$) fueron elección del estudiante conforme a la guía.

Alcance y verificación: La inteligencia artificial no se usó para elaborar el informe completo ni para generar resultados: se usó como apoyo de redacción, verificación y formato, como con las prácticas de la unidad anterior. El estudiante revisó los textos, comprobó que las cifras y los comentarios coincidieran con sus propios resultados y es responsable del contenido final. Dado que las herramientas de IA pueden cometer errores, las afirmaciones sobre las pruebas estadísticas se contrastaron con la guía de la práctica y con la bibliografía citada.

10. Bibliografía / Referencias

1. J. O. Guamán, “Guía de Actividades Práctico-Experimentales Nro. 006: Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov,” FEIRNNR, Carrera de Computación, Univ. Técnica del Norte, Ibarra, Ecuador, 2026.
2. J. O. Guamán, “Generadores de números pseudoaleatorios,” diapositivas de la Semana 5, Simulación, FEIRNNR – Carrera de Computación, 2026.
3. J. O. Guamán, “Pruebas estadísticas de uniformidad e independencia,” diapositivas de la Semana 6, Simulación, FEIRNNR – Carrera de Computación, 2026.
4. W. H. Press *et al.*, “Numerical Recipes: The Art of Scientific Computing,” 3rd ed., Cambridge Univ. Press, 2007, cap. 7 (generadores de números aleatorios).
5. A. M. Law, “Simulation Modeling and Analysis,” 5th ed., McGraw-Hill, 2015, cap. 8 (verificación y validación de generators pseudoaleatorios).

11. Anexos

Recursos de apoyo para cada una de las secciones anteriores:

- Código fuente Python: `python/` (`lcg.py`, `pruebas.py`, `main.py`, `figures.py`, `check_parity.py`).
- Código fuente Java: `java/src/main/java/ape6/` (`Lcg.java`, `Pruebas.java`, `Main.java`).
- Resultados numéricos: `results/stats_python.json`, `results/stats_java.json`, `results/observed_*_*.csv`.
- Secuencias de verificación: `results/parity_*_python.csv`, `results/parity_*_java.csv`.
- Figuras: `figures/`.